Application design of sustainable workloads on Azure

When building new or updating existing applications, it's crucial to consider how the solution will impact the climate and if there are ways to improve and optimize. Learn about considerations and recommendations to optimize your code and applications for a more sustainable application design.

> **Important**
>
> This article is part of the Azure Well-Architected sustainable workload series. If you aren't familiar with this series, we recommend you start with what is a sustainable workload?

Code efficiency

Demands on applications can vary, and it's essential to consider ways to stabilize the utilization to prevent over- or underutilization of resources, which can lead to unnecessary energy spills.

Evaluate moving monoliths to a microservice architecture

Monolithic applications usually scale as a unit, leaving little room to scale only the individual components that may need it.

Green Software Foundation alignment: Energy efficiency, Hardware efficiency

Recommendation:

- Evaluate the microservice architecture guidance.
- A microservice architecture allows for scaling of only the necessary components during peak load; ensuring idle components are scaled down or in. Additionally, it may reduce the overhead and resources required for deploying monolithic applications.
- Consider this tradeoff: While reducing the compute resources required, you may increase the amount of traffic on the network, and the complexity of the application may increase significantly.
- Consider this other tradeoff: Moving to microservices can result in extra deployment overhead with numerous similarities in deployment pipelines. Carefully consider the

required deployment resources for monolithic versus microservice architectures.
- Additionally, read about containerizing monolithic applications.

Improve API efficiency

Many modern cloud applications are designed to transact many messages between services and components asynchronously. Consider the format used to encode the payload data. How much information does your application need to communicate, and is there room to reduce the chattiness?

Green Software Foundation alignment: Energy efficiency

Recommendation:

- Learn about the chatty I/O antipattern to better understand how a large number of requests can impact performance and responsiveness.
- Improve the reliability and reduce unnecessary load to your systems. Implement advanced request throttling with API Management.
- Minimize the amount of data the application returns from requests by being selective and encoding the messages. See message encoding considerations.
- Cache responses to avoid reprocessing the same type of information from the backend system unless necessary. See caching in Azure API Management.

Ensure backward software compatibility to ensure it works on legacy hardware

Consider how applications render information. Does the application need to critically serve everything in the highest quality, resulting in higher bandwidth and processing? Is there room for reducing the quality of components in the UI to serve sustainability goals better?

Green Software Foundation alignment: Hardware efficiency

Recommendation:

- Support more end-user consumer devices, like older browsers and operating systems. This backward compatibility improves hardware efficiency by reusing existing hardware instead of requiring a hardware upgrade for the solution to work.
- Consider this tradeoff: If the most recent software updates have significant performance improvements, using older software versions may not be more efficient.

Leverage cloud native design patterns

Learning about cloud-native design patterns is helpful for building applications, whether they're hosted on Azure or running elsewhere. Optimizing the performance and cost of your cloud application will also reduce its resource utilization, hence its carbon emissions.

Green Software Foundation alignment: Energy efficiency, Hardware efficiency

Recommendation:

- Leverage cloud-native design patterns when writing or updating applications.

Consider using circuit breaker patterns

Consider evaluating and preventing applications from performing operations that are likely to fail. Repeated failures can lead to overhead and unnecessary processing that you can avoid with proper design patterns.

Green Software Foundation alignment: Energy efficiency

Recommendation:

- A circuit breaker can act as a proxy for operations that might fail and should monitor the number of recent failures that have occurred and use that information to decide whether to proceed.
- Study the Circuit Breaker pattern, and then consider how you can implement the Circuit Breaker patterns to your applications.
- Consider using Azure Monitor to monitor failures and set up alerts.

Optimize code for efficient resource usage

Applications deployed using inefficient code may result in an inherent impact on sustainability.

Green Software Foundation alignment: Energy efficiency, Hardware efficiency

Recommendation:

- Reduce CPU cycles and the number of resources you need for your application.
- Use optimized and efficient algorithms and design patterns.
- Consider the Don't repeat yourself (DRY) principle.

Optimize for async access patterns

Demands on applications can vary, and it's essential to consider ways to stabilize the utilization to prevent over- or underutilization of resources, which can lead to unnecessary energy spills.

Green Software Foundation alignment: Energy efficiency

Recommendation:

- Queue and buffer requests that don't require immediate processing, then process in batch. Designing your applications in this way helps achieve a stable utilization and helps flatten consumption to avoid spiky requests.
- Read about optimizing for async access patterns.

Evaluate server-side vs. client-side rendering

Determine whether to render on the server-side or client-side when building applications with a UI.

Green Software Foundation alignment: Energy efficiency, Hardware efficiency

Recommendation:

- Consider these benefits of server-side rendering:
  - When the server's power comes from less-polluting alternatives than the client's locale.
  - When the hardware on the server has better processing-energy ratios.
  - Can use centralized caching to reduce multiple unnecessary renders.
  - Reducing the number of browser-to-server round-trips can be particularly important when the client's device has a lossy link.
  - When the client devices are older and have slower CPUs. Users don't need to upgrade their devices to support a modern browser.

- Consider these benefits of client-side rendering:
  - When the end-user devices are more suitable, pushing the responsibility of rendering to the clients.
  - It's more efficient only to render what's needed and as requested, as opposed to rendering everything at least once.
  - There's no need for a server, as you can rely on static storage.
  - Browser caching is used on the clients.

Be aware of UX design for sustainability

Consider how the UX design of a workload impacts sustainability and determine what options exist for improving energy efficiency and reducing unnecessary network load, data processing, and compute resources.

Green Software Foundation alignment: Energy efficiency

Recommendation:

- Consider reducing the number of components to load and render on pages.
- Determine whether the application can render lower-resolution images and videos.
  - Don't render full-size images as thumbnails where the browser is doing the resizing.
  - Using full-size images as thumbnails or resized images will transfer more data, unnecessary network traffic, and additional client-side CPU usage due to image resizing and pre-rendering.
- Ensuring there are no unused pages will help minimize the UX design.
- Consider search and findability. Making it easier for users to find what they're looking for helps lower the amount of data stored and retrieved.
- Consider providing a lighter UI, using fewer resources and with a lower impact on sustainability, and provide users with an informed choice.
- Save energy by offering your apps and websites in dark mode, with dark backgrounds.
- Opt for using system fonts when possible to avoid forcing clients to download additional fonts, which causes more network load.

Update legacy code

Consider upgrading or deprecating legacy code if it's not running on modern cloud infrastructure or with the latest updates.

Green Software Foundation alignment: Hardware efficiency

Recommendation:

- Identify inefficient legacy code suited for modernization.
- Review if there are options to move to serverless or any of the optimized PaaS options.
- Consider this tradeoff: Updating old code that might end up being deprecated can consume valuable time.

Next step

Review the design considerations for the application platform.

Application platform

Last updated on 10/12/2022